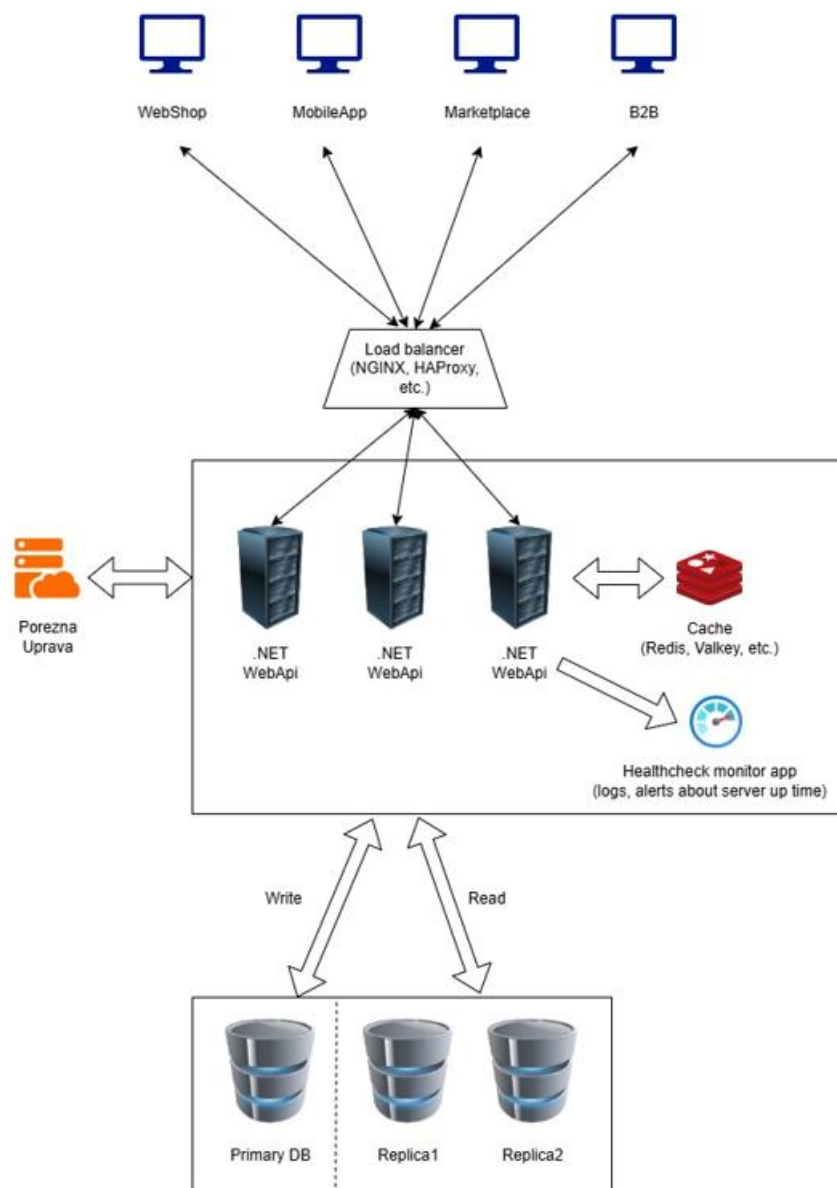


ShoppingApp



1. Arhitektura sustava

ShoppingApp je maloprodajna platforma koja podržava milijune korisnika dnevno. Kako bi se osigurala podrška za veliki promet koristit će se horizontalno skaliranje sustava i uravnoteživač opterećenja (eng. load balancer). Horizontalno skaliranje znači da će postojati više identičnih servera koji će posluživati korisnike. Serveri će biti implementirani u C#, odnosno Web API tehnologiji. Za odabir servera, koji će obraditi zahtjev od korisnika, koristit će se load balancer komponenta (za to već postoje provjerena rješenja poput NGINX, HAProxy itd.). Nakon što stigne zahtjev do jednog od servera, cilj je što brže te na siguran način obraditi ga. Budući da je u praksi pokazano da se većina zahtjeva odnosi na čitanje podataka iz baze, a tek manji broj pisanje podataka, server će implementirati CQRS (skraćenica za „Command and Query Responsibility Segregation“) dizajn. To

znači da će sustav imati dva kanala za obradu zahtjeva, jedan namijenjen čitanju, a drugi pisanju podataka. Samim time, potrebno je prilagoditi bazu podataka. Baza podataka također će biti horizontalno skalirana na način da postoji jedna primarna baza podataka za operacije pisanja te više replika primarne baze namijenjenih operacijama čitanja. Replike se periodički sinkroniziraju s primarnom bazom podataka. Ovim dizajnom se postiže da operacije čitanja, koje su uglavnom brže, ne ovise o sporim operacijama pisanja, što rezultira ubrzanjem cjelokupnog sustava. Za postizanje dodatnog ubrzanja operacija čitanja, koristit će se cache (priručna memorija koja privremeno sprema podatke). Za cache će se koristiti postojeće rješenje poput Redis, Valkey, ili neko drugo odgovarajuće rješenje.

2. Stabilnost sustava

Kako bi se osiguralo da sustav radi 24/7, serveri će implementirati HTTP endpoint koja će vraćati informaciju o tome je li server aktivan, tj. dostupan. Samu provjeru vršit će HealthcheckMonitoringApp koja će pingati svaki server periodički, te u slučaju da je neki server neaktivan, dizat će upozorenja. Administratori sustava će imati nadzornu ploču (eng. dashboard) preko koje će moći pratiti upozorenja i na njih reagirati.

3. Komunikacija s vanjskim servisima

Sustav će trebati komunicirati s vanjskim servisima poput Porezne uprave radi fiskalizacije. S tehničke strane, potrebno je osigurati da sustav može komunicirati cijelo vrijeme, odnosno da ne dođe do zagušenja (eng. socket exhaustion). To znači da pri implementaciji Web API servera bitno je obratiti pažnju na način instanciranja objekta HttpClient za komunikaciju s vanjskim serverom. Odnosno, da se koristi IHttpHttpClientFactory koji koristi pooling mehanizme za sprječavanje socket exhaustion.

4. Sigurnost i autentifikacija

Za sigurnost sustava, odnosno autentifikaciju i autorizaciju, koristit će se Microsoft Identity i JWT (JSON web token) tehnologija. Pomoću tehnologije Microsoft Identity korisnici će se moći registrirati i ulogirati. Za login će biti potreban email (jedan za svakog korisnika) i password. Passwordi će se u bazu spremati po najvišim sigurnosnim standardima (hashing i salt). Kada se korisnik ulogira u aplikaciju, za daljnju komunikaciju sa serverom upotrebljavat će access token. Access token će u sebi sadržavati informacije o korisniku kao što su identifikator korisnika i njegove uloge, preko kojih će server znati kojim resursima korisnik smije pristupiti. JWT predstavlja standardizirani format za siguran prijenos informacija između klijenta i servera, pri čemu se integritet i autentičnost tokena osiguravaju njegovim digitalnim potpisom.

5. Plan isporuke koda

Za verzioniranje koda koristit će se Git tehnologija s dva glavna brancha TEST i PROD. Za svaku funkcionalnost potrebno je imati zaseban feature branch. Nakon što je neki feature gotov, on se prvo merge-a u TEST branch, gdje će biti dostupan QA timu na testiranje. Prije svakog merge-a u TEST branch nužno je da i svaki programer sam istestira funkcionalnosti (provjeri sve rubne slučajeve) te da barem jedan kolega napravi code review. Nakon što je neki feature istestiran i spreman za produkciju, čeka se odobrenje od projekt managera za njegovo puštanje na produkciju, odnosno merge u PROD branch. PROD branch će se puštati na produkciju periodički (npr. svakih mjesec dana). Iznimka su hotfixevi, odnosno ispravak kritičnih greški na sustavu.